

Lineare Algebra - Übungen 3

Klaus Rheinberger, FH Vorarlberg

20. März 2025

Abgabe in ILIAS

- Alle PYTHON-Aufgaben gesamthaft in einem lauffähigen Jupyter Notebook, siehe Vorlage in ILIAS.
- Alle PAPIER-Aufgaben gesamthaft in einer PDF-Datei inkl. Name auf den gescannten Seiten.

1 Lesen, Studieren und Fragen Notieren (0 Punkte)

Lesen und studieren Sie vor der Bearbeitung der Aufgaben auf der LV-Homepage die Abschnitte [LGS/Methoden/Schreibweisen](#) bis inkl. [LGS/Methoden/Python](#) sowie die durchgerechneten [LGS/Beispiele](#). Notieren Sie sich Fragen und Unklarheiten, die Sie in der Übung besprechen möchten.

2 Gaußsches Eliminationsverfahren (2 + 2 = 4 Punkte)

1. PAPIER: Bestimmen Sie von Hand mit Hilfe des GEV, ob folgendes lineares Gleichungssystem nicht-triviale Lösungen hat und, falls ja, geben Sie diese an:

$$2x_1 - 5x_2 + 8x_3 = 0$$

$$-2x_1 - 7x_2 + x_3 = 0$$

$$4x_1 + 2x_2 + 7x_3 = 0$$

2. PYTHON: Überprüfen Sie Ihr Ergebnis am Computer, indem Sie
 1. die Koeffizientenmatrix A und die erweiterte Koeffizientenmatrix $(A|b)$ erstellen und ihre Ränge bestimmen.
 2. überprüfen, ob Ihre handschriftliche Lösung korrekt ist.

Lösung:

1. PAPIER: Das GEV liefert, dass das Gleichungssystem nicht-triviale Lösungen hat, nämlich alle Vielfachen von $x = \begin{pmatrix} -17 \\ 6 \\ 8 \end{pmatrix}$. Siehe Abbildung 1.
2. PYTHON:

$$\begin{array}{ccc|c}
 2 & -5 & 8 & 0 \\
 -2 & -7 & 1 & 0 \quad | +2_1 \\
 \hline
 4 & 2 & 7 & 0 \quad | -2 \cdot 2_1 \\
 \hline
 2 & -5 & 8 & 0 \\
 0 & -12 & 9 & 0 \quad | :3, \\
 \hline
 0 & 12 & -9 & 0 \quad | :3, +2_2 \\
 \hline
 2 & -5 & 8 & 0 \\
 0 & -4 & 3 & 0 \\
 \hline
 0 & 0 & 0 & 0
 \end{array}$$

$\text{rank}(A) = \text{rank}(A|b) = 2,$
 $n = 3$. Daher 1 frei wählbarer Parameter: x_3

$$\begin{aligned}
 & \rightarrow -4x_2 + 3x_3 = 0 \\
 & \quad x_2 = \frac{3}{4}x_3 \\
 & 2x_1 - 5x_2 + 8x_3 = 0 \\
 & 2x_1 - \frac{15}{4}x_3 + 8x_3 = 0 \\
 & 2x_1 = \frac{15-32}{4}x_3 \\
 & \quad x_1 = \frac{-17}{8}x_3
 \end{aligned}$$

Daher ∞ -viele Lösungen
 $x_1 = -\frac{17}{8}x_3$
 $x_2 = \frac{3}{4}x_3$
 x_3 frei wählbar.
 z.B. $x = (-17, 6, 8)$

Abbildung 1: Gaußsches Eliminationsverfahren

```

import numpy as np
import matplotlib.pyplot as plt

# matrix form Ax = b of the system of linear equations:
A = np.array([[ 2, -5, 8],
              [-2, -7, 1],
              [ 4,  2, 7]])
b = np.zeros((3, 1))

print(f"Rang(A) = {np.linalg.matrix_rank(A)}")
Ab = np.hstack((A, b))
print(f"Rang(A|b) = {np.linalg.matrix_rank(Ab)}")
# Ax = b hat also unendlich viele Lösungen, da Rang(A)=Rang(A|b) < Anzahl Variablen.

# Check if x = (-17, 6, 8) is a solution:
x = np.array([-17, 6, 8])
print(f"Ax = {A@x}") # should be zero

```

```

Rang(A) = 2
Rang(A|b) = 2
Ax = [0 0 0]

```

Endergebnis

Das Gleichungssystem hat nicht-triviale Lösungen, nämlich alle Vielfachen von $x = \begin{pmatrix} -17 \\ 6 \\ 8 \end{pmatrix}$.

3 GEV und Python (4 + 4 + 4 = 12 Punkte)

1. PAPIER: Lösen Sie folgende LGS mit dem GEV. Gebe Sie jeweils auch den Rang der Koeffizientenmatrix und der erweiterten Koeffizientenmatrix an, und schließen Sie daraus auf die Lösungsstruktur.
2. PYTHON: Überprüfen Sie Ihr Ergebnisse am Computer, indem Sie jeweils
 1. die Koeffizientenmatrix A und die erweiterte Koeffizientenmatrix $(A|b)$ erstellen und ihre Ränge bestimmen.
 2. überprüfen, ob Ihre handschriftliche Lösung korrekt ist.

Beispiel A:

$$\begin{aligned}x_1 + x_2 + x_3 &= 1 \\x_1 + 2x_2 + 2x_3 &= 1 \\x_1 + 2x_2 + 3x_3 &= 1\end{aligned}$$

Beispiel B:

$$\begin{aligned}-x_1 + 3x_2 - 2x_3 &= 1 \\-x_1 + 4x_2 - 3x_3 &= 0 \\-x_1 + 5x_2 - 4x_3 &= 0\end{aligned}$$

Beispiel C:

$$\begin{aligned}-x_1 + 3x_2 - 2x_3 &= 4 \\-x_1 + 4x_2 - 3x_3 &= 5 \\-x_1 + 5x_2 - 4x_3 &= 6\end{aligned}$$

Lösung:

Beispiel A:

Koeffizientenmatrix A:

```
[[1 1 1]
 [1 2 2]
 [1 2 3]]
```

Rechte Seite b:

```
[[1]
 [1]
 [1]]
```

Erweiterte Koeffizientenmatrix (A|b):

```
1  1  1 | 1
1  2  2 | 1
1  2  3 | 1
```

Erweiterte Koeffizientenmatrix nach 2 Umformungen:

```
1  1  1 | 1
```

```

0  1  1 | 0
0  1  2 | 0
Erweiterte Koeffizientenmatrix nach 3 Umformungen:
1  1  1 | 1
0  1  1 | 0
0  0  1 | 0
Eindeutige Lösung nach Rücksubstitutionen: x = [1. 0. 0.]

```

Überprüfung in Python:

```

A = np.array([[1, 1, 1],
              [1, 2, 2],
              [1, 2, 3]])
b = np.array([[1], [1], [1]])
Ab = np.hstack((A, b))
rA = np.linalg.matrix_rank(A)
rAb = np.linalg.matrix_rank(Ab)
print(f"Rang von A ist {rA}, und Rang von (A|b) ist {rAb}.")

x = np.array([1, 0, 0])
test = A@x
print(f"Lösungsüberprüfung: Ax = {test}")

sol = np.linalg.solve(A, b)
print(f"Lösung mittels np.linalg.solve:\n x = {sol}")

```

```

Rang von A ist 3, und Rang von (A|b) ist 3.
Lösungsüberprüfung: Ax = [1 1 1]
Lösung mittels np.linalg.solve:
x = [[1.]
      [0.]
      [0.]]

```

Beispiel B:

```

Koeffizientenmatrix A:
[[-1  3 -2]
 [-1  4 -3]
 [-1  5 -4]]
Rechte Seite b:
[[1]
 [0]
 [0]]
Erweiterte Koeffizientenmatrix (A|b):
-1  3 -2 | 1
-1  4 -3 | 0
-1  5 -4 | 0
Erweiterte Koeffizientenmatrix nach 1. Umformung:
-1  3 -2 | 1
0  1 -1 |-1

```

```

-1  5 -4 | 0
Erweiterte Koeffizientenmatrix nach 2. Umformung:
-1  3 -2 | 1
  0  1 -1 |-1
  0  2 -2 |-1
Erweiterte Koeffizientenmatrix nach 3. Umformung:
-1  3 -2 | 1
  0  1 -1 |-1
  0  0  0 | 1
LGS hat keine Lösung.

```

Überprüfung in Python:

```

A = np.array([[ -1,  3, -2],
              [ -1,  4, -3],
              [ -1,  5, -4]])
b = np.array([[1], [0], [0]])
Ab = np.hstack((A, b))
rA = np.linalg.matrix_rank(A)
rAb = np.linalg.matrix_rank(Ab)
print(f"Rang von A ist {rA}, und Rang von (A|b) ist {rAb}.")

```

Rang von A ist 2, und Rang von (A|b) ist 3.

Beispiel C:

```

Koeffizientenmatrix A:
[[ -1  3 -2]
 [ -1  4 -3]
 [ -1  5 -4]]
Rechte Seite b:
[[4]
 [5]
 [6]]
Erweiterte Koeffizientenmatrix (A|b):
-1  3 -2 | 4
-1  4 -3 | 5
-1  5 -4 | 6
Erweiterte Koeffizientenmatrix nach 1. Umformung:
-1  3 -2 | 4
  0  1 -1 | 1
-1  5 -4 | 6
Erweiterte Koeffizientenmatrix nach 2. Umformung:
-1  3 -2 | 4
  0  1 -1 | 1
  0  2 -2 | 2
Erweiterte Koeffizientenmatrix nach 3. Umformung:
-1  3 -2 | 4
  0  1 -1 | 1
  0  0  0 | 0

```

LGS hat unendliche viele Lösungen.

x_3 frei wählbar

$x_2 = 1 + x_3$

$x_1 = -1 + x_3$

Überprüfung in Python:

```
A = np.array([[ -1,  3, -2],
              [-1,  4, -3],
              [-1,  5, -4]])
b = np.array([[4], [5], [6]])
Ab = np.hstack((A, b))
rA = np.linalg.matrix_rank(A)
rAb = np.linalg.matrix_rank(np.hstack((A, b)))
print(f"Rang von A ist {rA}, und Rang von (A|b) ist {rAb}.")

# Eine mögliche Lösung:
x_3 = 1
x = np.array([-1 + x_3, 1 + x_3, x_3])
test = A @ x
print(f'Lösungsüberprüfung: Für x_3 = 1 ist Ax = {test}')
```

Rang von A ist 2, und Rang von (A|b) ist 2.

Lösungsüberprüfung: Für $x_3 = 1$ ist $Ax = [4 \ 5 \ 6]$

Endergebnis

- Beispiel A: Rang von A ist 3, und Rang von $(A|b)$ ist 3. Das LGS hat die eindeutige Lösung $x = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$.
- Beispiel B: Rang von A ist 2, und Rang von $(A|b)$ ist 3, LGS hat keine Lösung.
- Beispiel C: Rang von A ist 2, und Rang von $(A|b)$ ist 2, LGS hat unendlich viele Lösungen mit z. B. x_3 frei wählbar, dann $x_2 = 1 + x_3$ und $x_1 = -1 + x_3$.

4 Existenz und Eindeutigkeit (2 Punkte)

Gegeben ist das lineare Gleichungssystem:

$$2x_1 - 4x_2 - 2x_3 = b_1$$

$$-5x_1 + x_2 + x_3 = b_2$$

$$7x_1 - 5x_2 - 3x_3 = b_3$$

PYTHON: Hat das lineare Gleichungssystem für alle Vektoren b eine eindeutige Lösung? Begründen Sie Ihre Antwort nach einer passenden Rechnung in Python in einem Python-Kommentar.

Lösung:

```
A = np.array([[ 2, -4, -2],
              [-5,  1,  1],
              [ 7, -5, -3]])
print(f"rank(A) = {np.linalg.matrix_rank(A)}")
```

```
rank(A) = 2
```

Das lineare Gleichungssystem $Ax = b$ hat nicht für alle Vektoren b eine eindeutige Lösung, denn der Rang von A ist nur 2 und somit nicht voll. Ein voller Rang wäre ein Rang von 3. Daher gibt es rechte Seiten b , sodass die erweiterte Koeffizientenmatrix $(A|b)$ den Rang 3 hat, die Koeffizientenmatrix A aber nur den Rang 2.

Endergebnis

Das lineare Gleichungssystem hat nicht für alle Vektoren b eine eindeutige Lösung.

5 Parabel durch drei Punkte (2 + 2 + 2 = 6 Punkte)

1. PAPIER: Stellen Sie ein LGS zur Bestimmung der Koeffizienten a, b, c der Parabel (= quadratische Funktion)

$$y(x) = a + bx + cx^2$$

auf, welche durch die Punkte $(1, 1)$, $(2, 2)$ und $(3, 0)$ geht. Lösen Sie anschließend das LGS mit dem GEV.

2. PYTHON: Lösen Sie das LGS, und stellen Sie die gefundene Parabel grafisch dar.
3. PAPIER: Wie müssten die Punkte liegen, damit das LGS keine Lösung hat? Wie müssten die Punkte liegen, damit das LGS unendlich viele Lösungen hat? Begründen Sie jeweils Ihre Antwort.

Lösung: Aufstellen der 3 Gleichungen für die 3 Punkte für $y(x) = a + bx + cx^2$:

- Punkt $(1, 1)$ einsetzen: $1 = a + b + c$
- Punkt $(2, 2)$ einsetzen: $2 = a + 2b + 4c$
- Punkt $(3, 0)$ einsetzen: $0 = a + 3b + 9c$

Erweiterte Koeffizientenmatrix:

```
1  1  1 | 1
1  2  4 | 2
1  3  9 | 0
```

Erweiterte Koeffizientenmatrix nach 1. Umformung:

```
1  1  1 | 1
0  1  3 | 1
1  3  9 | 0
```

Erweiterte Koeffizientenmatrix nach 2. Umformung

```
1  1  1 | 1
0  1  3 | 1
0  2  8 |-1
```

Erweiterte Koeffizientenmatrix nach 3. Umformung:

```
1  1  1 | 1
0  1  3 | 1
```

0 0 2 |-3
Lösung nach Rücksubstitution: a = -3.00, b = 5.50, c = -1.50

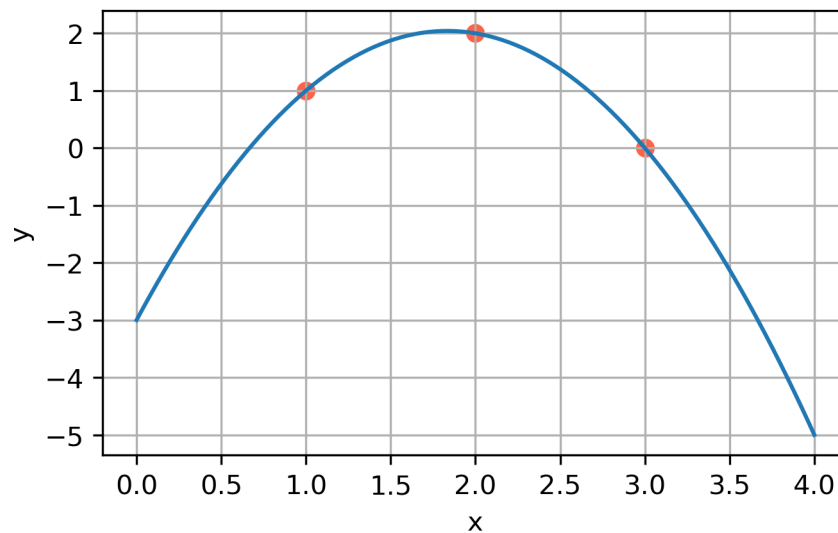
```
A = np.array([[1, 1, 1],
              [1, 2, 4],
              [1, 3, 9]])
b = np.array([1, 2, 0])
sol = np.linalg.solve(A, b)
print(f"Lösung mittels np.linalg.solve:\n sol = {sol}")

a = sol[0]
b = sol[1]
c = sol[2]

x = np.linspace(0, 4, 101)
y = a + b*x + c*x**2

plt.figure(figsize=(5, 3))
plt.scatter([1, 2, 3], [1, 2, 0], marker='o', color='tomato')
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.grid()
```

Lösung mittels np.linalg.solve:
sol = [-3. 5.5 -1.5]



Parabel: $y(x) = -3 + 5.5x - 1.5x^2$.

Endergebnis

Parabel: $y(x) = -3 + 5.5x - 1.5x^2$.